

Evaluating the Memory/Latency Boundary of Adaptive Symbol-Table Name Representations in Embedded Workloads

Shubhi Singh
SCOPE, VIT Vellore
Vellore, India

Adhyan Jain
SCOPE, VIT Vellore
Vellore, India

Akshith Venkataramana
SCOPE, VIT Vellore
Vellore, India

Arjuman
SCOPE, VIT Vellore
Vellore, India

Abstract—Symbol tables are a core compiler data structure whose physical memory footprint and lookup latency directly govern execution efficiency under severe resource constraints. Modern 64-bit systems rely on `std::unordered_map` with Short String Optimization (SSO), storing small identifiers (≤ 15 bytes) without secondary heap allocations. In this paper, we investigate the fundamental memory and latency trade-offs of adaptive symbol-table representations in embedded compiler and runtime environments. We formulate the primary research question: *Under what workload conditions can adaptive symbol-table name representations overcome the structural memory efficiency of an embedded conventional hash-table baseline while satisfying a latency constraint?*

We present **SymTabV3**, an adaptive scope-aware symbol table that dynamically assigns symbols to `INLINE`, `INTERNEDED`, or front-coded `COMPRESSED` representations with zero-allocation stack buffer decoding. We derive a closed-form analytical break-even model: for identifiers longer than the SSO threshold ($L > 15$ B), adaptive storage beats Conventional hash tables when the live duplication ratio satisfies $k > k_{\text{breakeven}} = (2L + 86)/(L + 13)$. We evaluate our model using physical allocator heap tracking (`malloc_usable_size`) across four real embedded workloads (FREERTOS, ARDUINO, ZEPHYR, ESP-IDF), 20 host software corpora, and controlled synthetic experiments (Workloads A–F).

On the large ZEPHYR workload (2.6M events, 228,739 unique symbols), **SymTabV3** reduced physically measured final heap by 20.2% (13.52 MB) relative to the embedded conventional baseline (53.39 MB vs 66.91 MB final heap; 57.89 MB vs 76.37 MB peak heap). This reduction was achieved with sub-microsecond median lookup latency ($p_{50} = 0.066 \mu\text{s}$), while the p_{95} latency increased from $0.125 \mu\text{s}$ to $0.228 \mu\text{s}$ ($1.824\times$). Thus, the architecture demonstrates a measurable RAM advantage in the large-scale embedded regime, but does not satisfy the $1.25\times p_{95}$ latency constraint due to measured reconstruction/copy overhead. Additionally, we evaluate **SymTabV4** as a disclosed negative result, showing that side-table container overheads (T_{table}) erode core scalar field savings across real identifier mixes.

Index Terms—Symbol Table, Compiler Memory Management, Embedded Systems, Short String Optimization, String Interning, Front Coding, Data Structure Break-Even Boundary, Empirical Evaluation.

I. INTRODUCTION

Compilers and language interpreters repeatedly create, resolve, and reclaim symbolic bindings during lexical, syntactic, and semantic analysis [1]. A symbol table maps identifier names to binding metadata across nested lexical scopes. In resource-constrained embedded systems—such as microcon-

troller toolchains, real-time operating systems (RTOS), and on-device interpreters—available RAM is severely limited.

In desktop and server host compilers (e.g., LLVM [2]), standard C++ string implementations rely on Short String Optimization (SSO) [3], [4]. Identifiers up to 15 bytes are stored directly inside the 24-byte string structure, avoiding heap allocations. Consequently, conventional hash maps (`std::unordered_map<std::string, SymbolMeta>`) pay zero secondary heap string allocation cost for short names.

In embedded environments, however, secondary heap overheads and container metadata can consume a significant fraction of total memory. While string interning [5], [6] and front-coded block compression [7], [8] reduce string redundancy, their physical RAM savings and tail-latency costs under realistic scope lifecycles have lacked rigorous physical allocator measurement.

In this work, we formulate and address the following central research question:

Under what workload conditions can adaptive symbol-table name representations overcome the structural memory efficiency of an embedded conventional hash-table baseline while satisfying a latency constraint?

We make the following main contributions:

- 1) **Adaptive Architecture (SymTabV3)**: We present a C++ symbol table architecture featuring scope-lifetime slot recycling, 3-tier representation routing (`INLINE`, `INTERNEDED`, `COMPRESSED`), 1-byte hash fingerprints, and zero-allocation stack buffer decoding.
- 2) **Analytical Cost & Break-Even Model**: We derive closed-form memory equations proving that for $L \leq 15$ B no positive break-even live duplication ratio k exists, whereas for $L > 15$ B, interning breaks even when $k > (2L + 86)/(L + 13)$.
- 3) **Embedded Memory Advantage**: On the large ZEPHYR RTOS workload, **SymTabV3** achieves a 20.2% (13.52 MB) physical final heap reduction (and 24.2% peak heap reduction) over an optimized embedded conventional baseline.

- 4) **Quantified Memory/Latency Trade-off:** We demonstrate sub-microsecond median lookups ($p_{50} = 0.066 \mu s$) on ZEPHYR, while reporting that p_{95} tail latency increases to $0.228 \mu s$ (vs $0.125 \mu s$ for the embedded conventional baseline), failing the $1.25 \times p_{95}$ gate due to measured reconstruction/copy overhead.
- 5) **Disclosed Negative Architectural Result (SymTabV4):** We present SymTabV4 (representation-conditional metadata) as a disclosed negative result, showing that sparse side-table allocation floors (T_{table}) outweigh core struct savings across real representation mixes.
- 6) **Controlled Synthetic & 20-Corpus Host Validation:** We confirm theoretical predictions via synthetic microbenchmarks A–F ($k \geq 3.33$ at $L = 32 B$) and 20 host software corpora.

II. BACKGROUND AND MOTIVATION

A. Conventional Symbol Tables and SSO

A conventional symbol table uses a hash map from string keys to symbol attributes. In 64-bit systems, a `std::string` instance occupies 32 bytes (or 24 bytes under `libc++`), consisting of a pointer, length, and capacity/inline buffer. When identifier length $L \leq 15$ bytes, characters reside inside the inline buffer (SSO). For $L > 15$ bytes, `std::string` allocates a heap buffer of size $L+1$ bytes plus allocator header overhead $H \approx 16 B$.

In host standard libraries, `std::unordered_map` node allocations incur node overhead ($S_{node} \approx 32 B$). For short names ($L \leq 15 B$), Conventional pays only metadata and node footprint:

$$M_{conv_host}(k, L) \approx k \cdot (S_{meta} + S_{node}) \quad \text{for } L \leq 15 B \quad (1)$$

For embedded targets, we construct `EmbeddedConventional`, a compact baseline combining 16-byte fixed entries, a contiguous byte arena, and an open-addressing slot index.

B. String Interning

String interning maintains a global pool of unique string instances. Each symbol record stores a 4-byte pool index. However, interning incurs two fixed costs: 1) a global pool vector; 2) a lookup map (`poolLookup_`) paying node overhead $S_{pool_node} \approx 68 B$ per unique name.

The memory for k live occurrences of a unique name of length L under interning is:

$$C_{pool}(L) \approx 2L + 86 \quad (2)$$

$$M_{int}(k, L) \approx k \cdot C_{idx} + C_{pool}(L) \quad (3)$$

where $C_{idx} \approx 28 B$ is the index slot overhead.

III. RELATED WORK

A. Compiler Symbol Tables

Scope management and binding resolution have been studied since early structured language compilers [1], [13], [14]. Traditional scope stacks push a hash map on scope entry and pop it on exit. LLVM’s `IdentifierTable` [2] utilizes a global string map backed by a bump-pointer allocator (`BumpPtrAllocator`), prioritizing allocation throughput but allowing memory to grow monotonically until compilation completes.

B. String Compression and Dictionary Search

String interning is standard in runtimes such as Java HotSpot [5] and CPython [6]. For text indexing, front-coded block compression [7]–[9] stores lexicographically shared prefixes relative to periodic anchor strings. Succinct data structures [10], [11] and minimal perfect hash functions (e.g., CHD [12]) achieve optimal static byte density but cannot handle dynamic lexical scope entry and exit.

C. Cache-Conscious Hash Tables

Open-addressing structures such as Robin Hood hashing [15], Cuckoo hashing [16], HAT-trie [17], and burst tries [18] optimize L1/L2 cache locality. SymTabV3 complements these by integrating scope-lifetime slot recycling, 3-tier representation routing, 1-byte hash fingerprints, and zero-allocation stack buffer decoding.

IV. ADAPTIVE SYMBOL-TABLE ARCHITECTURE

A. SymTabV3 Core Design

SymTabV3 organizes entries into a scope-managed slot arena. Each slot pays a 32-byte `PackedEntry` containing scope ID, access counters, representation tags, and a 12-byte payload union:

- **INLINE:** Identifiers ≤ 12 bytes are stored inside the slot union (`inlineBytes[12]`), requiring zero secondary allocations.
- **INTERNED:** Duplicate or hot symbols store a 4-byte index into a global pool.
- **COMPRESSED:** Long, cold symbols are front-coded in blocks of size $B = 32$ with anchor interval $A = 8$.

Scope exit events (`exitScope()`) recycle slot indices immediately into a LIFO free-list, preventing fragmentation. To avoid unnecessary decompression during lookup, SymTabV3 checks a 1-byte hash fingerprint array (`fp8`), rejecting non-matching candidates in $O(1)$ time.

B. Zero-Allocation Compressed Path Optimization

To minimize front-coded decode latency, SymTabV3 implements zero-allocation stack buffer decoding (`decodeToBuffer`). Rather than instantiating temporary `std::string` objects or executing substring concatenations (`cur.substr(0, len) + suffix`), prefix and suffix segments are copied directly into a stack-allocated buffer (`char stackBuf[512]`) via fast `memcpy`. Furthermore,

anchor indices are computed in $O(1)$ time (start = slot – (slot (mod A))), eliminating backward scanning loops. Name equality (`nameEquals`) performs direct in-place byte comparisons against `stackBuf`.

C. SymTabV4: Disclosed Negative Result

To test whether removing hot/cold metadata from `INLINE` entries improves byte density, we designed `SymTabV4`. `SymTabV4` reduces `PackedEntry` from 32 B to 24 B (`CoreEntry`) by moving scalar metadata (`accessCount`, `lastAccessEpoch`) to a sparse side table (`HotMetaTable`). Both node-based and Robin Hood open-addressing side-table variants were implemented and differentially verified against `SymTabV3` under `ASan/UBSan` across 200,000 trace steps.

V. ANALYTICAL COST AND BREAK-EVEN MODEL

Equating $M_{\text{int}}(k, L) < M_{\text{conv}}(k, L)$ yields the theoretical break-even condition:

$$k > k_{\text{breakeven}}(L) = \frac{C_{\text{pool}}(L)}{S_{\text{meta}} + S_{\text{str}}(L) - C_{\text{id}}(L)} \quad (4)$$

For short names ($L \leq 15$ B), Conventional string storage $S_{\text{str}}(L) = 0$ due to SSO. Substituting $S_{\text{str}}(L) = 0$ yields:

$$k > \frac{2L + 86}{24 - 28} = \frac{2L + 86}{-4} < 0 \quad (5)$$

Because the denominator is negative, **no positive live duplication ratio k exists for $L \leq 15$ B**. Interning cannot outperform Conventional hash tables on short SSO identifiers.

For long names ($L > 15$ B), Conventional pays heap allocation $S_{\text{str}}(L) = L + 1 + H \approx L + 17$. The break-even equation becomes:

$$k_{\text{breakeven}}(L) = \frac{2L + 86}{24 + (L + 17) - 28} = \frac{2L + 86}{L + 13} \quad (6)$$

As $L \rightarrow 15^+$, $k_{\text{breakeven}} \rightarrow 4.14$. As $L \rightarrow \infty$, $k_{\text{breakeven}} \rightarrow 2.0$. For $L = 32$ B, $k_{\text{breakeven}} = 150/45 \approx 3.33$.

For `SymTabV4` to beat `SymTabV3`, core-struct savings across `INLINE` entries must exceed the side table’s allocation floor (T_{table}) and entry cost ($S_{\text{hotmeta_marginal}} \approx 34\text{--}57$ B):

$$k_{\text{inline}} \cdot 8 \text{ B} > k_{\text{hot}} \cdot S_{\text{hotmeta_marginal}} + T_{\text{table}}(N_{\text{hot}}) \quad (7)$$

Because real software codebases maintain 10%–70% non-`INLINE` symbols, k_{hot} is non-zero, causing side-table overheads to exceed core scalar field savings.

VI. EXPERIMENTAL METHODOLOGY

A. Physical Allocator Heap Accounting

All heap measurements use a physical allocator interceptor (`include/heap_counter.hpp`) overriding global operator `new` and operator `delete`. The interceptor queries `malloc_usable_size(p)` to record true physical heap allocations, accounting for system page alignment and chunk rounding.

B. Workload Suite

We evaluate three distinct workload categories:

- 1) **Real Embedded Workloads:** Four real-world embedded software event traces: `FREERTOS` (227k events, 10,386 unique symbols), `ARDUINO` (94k events, 11,000 unique symbols), `ZEPHYR` (2.6M events, 228,739 unique symbols), and `ESP-IDF` (2.2M events, 231,075 unique symbols).
- 2) **Host Software Corpora Characterization:** 20 real software codebases evaluated on 64-bit host systems to assess identifier length distributions and SSO fractions.
- 3) **Synthetic Microbenchmarks (Workloads A–F):** Controlled experiments varying identifier length $L \in \{4 \dots 64\}$ B, duplication ratio $k \in \{1 \dots 20\}$, scope depth, and access skew ($N = 30$ seeds).

Throughout this paper, we explicitly distinguish **peak heap** (maximum RAM allocated during execution) from **final heap** (residual RAM after processing complete event traces).

VII. EMPIRICAL RESULTS

A. Primary Embedded Evaluation

Table I presents physical allocator heap measurements, lookup latencies, representation counts, and gate status across the four embedded workloads.

B. Physical Memory Victory on Zephyr

On the large `ZEPHYR` workload (2.6M events, 228,739 unique symbols), `SymTabV3` achieves a clear physical allocator memory victory. `SymTabV3` reduces final heap consumption by ****20.2% (13.52 MB)**** relative to `EmbeddedConventional` (53.39 MB vs 66.91 MB final heap) and by ****34.3% (27.83 MB)**** relative to `Interned` (53.39 MB vs 81.22 MB final heap). Peak heap memory is similarly reduced by 24.2% (57.89 MB vs 76.37 MB).

Representation routing on `ZEPHYR` assigns 213,614 symbols (93.4%) to `INLINE`, 5,372 (2.3%) to `INTERNEDED`, and 9,753 (4.3%) to front-coded `COMPRESSED`.

C. Quantified Latency Trade-off and Reconstruction Overhead

The zero-allocation compressed-path optimization reduced `ZEPHYR` p_{95} latency from 0.547 μs down to 0.228 μs without altering memory footprint. Median lookup latency remains sub-microsecond ($p_{50} = 0.066 \mu\text{s}$).

However, comparing `SymTabV3` p_{95} (0.228 μs) against `EmbeddedConventional` p_{95} (0.125 μs) yields a ratio of ****1.824 $\times$ ****, failing the $1.25 \times p_{95}$ latency constraint.

This latency gap is driven by measured reconstruction/copy overhead. During execution, 8.81% of total lookup operations hit `COMPRESSED` entries. Reconstructing front-coded members requires copying multiple prefix and suffix segments across block members (mean depth 3.31 steps to anchor, $p_{95} = 7$ steps). Even with zero dynamic memory allocation, copying multiple memory slices introduces measured reconstruction/copy overhead that prevents satisfying the $1.25 \times$ gate.

TABLE I
CANONICAL PHYSICAL ALLOCATOR-MEASURED MEMORY AND LOOKUP LATENCIES ACROSS FOUR EMBEDDED WORKLOADS.

Workload	Implementation	Unique Syms	Peak Heap	Final Heap	p_{50} (μs)	p_{95} (μs)	p_{99} (μs)	Mean (μs)	Final RAM vs EmbConv	p
FreeRTOS	ConventionalHost	10,386	3.99 MB	3.74 MB	0.072	0.189	0.295	0.089	–	
	EmbeddedConventional	10,386	4.23 MB	3.89 MB	0.051	0.094	0.149	0.057	Baseline	
	Interned	10,386	5.17 MB	5.09 MB	0.072	0.143	0.203	0.087	+30.8%	
	SymTabV3 (Optimized)	10,386	4.78 MB	4.72 MB	0.071	0.277	0.445	0.101	+21.2%	
Arduino	ConventionalHost	11,000	2.42 MB	2.42 MB	0.080	0.192	0.276	0.093	–	
	EmbeddedConventional	11,000	2.31 MB	2.27 MB	0.046	0.089	0.130	0.051	Baseline	
	Interned	11,000	3.70 MB	3.70 MB	0.070	0.135	0.195	0.079	+63.0%	
	SymTabV3 (Optimized)	11,000	2.99 MB	2.99 MB	0.061	0.171	0.356	0.077	+31.7%	
Zephyr	ConventionalHost	228,739	50.92 MB	47.76 MB	0.096	0.339	0.680	0.133	–	
	EmbeddedConventional	228,739	76.37 MB	66.91 MB	0.050	0.125	0.225	0.060	Baseline	
	Interned	228,739	81.52 MB	81.22 MB	0.082	0.233	0.499	0.106	+21.4%	
	SymTabV3 (Optimized)	228,739	57.89 MB	53.39 MB	0.066	0.228	0.481	0.092	-20.2%	
ESP-IDF	ConventionalHost	231,075	73.28 MB	65.65 MB	0.103	0.255	0.536	0.123	–	
	EmbeddedConventional	231,075	75.77 MB	67.82 MB	0.061	0.236	0.374	0.086	Baseline	
	Interned	231,075	100.21 MB	92.95 MB	0.090	0.216	0.573	0.113	+37.1%	
	SymTabV3 (Optimized)	231,075	89.36 MB	81.91 MB	0.079	0.297	0.712	0.113	+20.8%	

Quantitatively, SymTabV3 trades $0.103 \mu s$ of added p_{95} latency for 13.52 MB of physical final heap saved (131.26 MB saved per μs of added p_{95} latency).

D. Disclosed Negative Result (SymTabV4)

SymTabV4 (representation-conditional side-table metadata) failed to beat SymTabV3 on 19/20 real software corpora and lost to Conventional on 20/20. Sparse side-table container allocations (T_{table}) plus marginal entry costs ($\sim 34\text{--}57\text{ B}$) exceed the 8 B core-struct savings across real identifier mixes.

E. Controlled Synthetic Microbenchmarks (Workloads A–F)

Synthetic experiments D1 and D2 confirm our analytical model. For $L = 32\text{ B}$, interning and SymTabV3 beat Conventional on peak heap at $k \geq 3$ (1.09 MB vs 1.16 MB) and on final heap at $k \geq 4$ (0.70 MB vs 1.44 MB), validating the theoretical break-even threshold $k_{\text{breakeven}} = 3.33$.

F. Supporting Host-Side 20-Corpus Characterization

On 64-bit host systems, ConventionalHost (`std::unordered_map`) achieves the lowest physical heap footprint on 20/20 real software corpora. Real host codebases are heavily dominated by short SSO identifiers (59%–76%) and low live duplication ($k < 1.5$), placing them strictly inside Conventional’s optimal memory regime.

VIII. DISCUSSION AND WORKLOAD BOUNDARIES

Our findings establish clear application boundaries for symbol table architectures:

- 1) **Host Compilers & Short Workloads:** When short SSO identifiers dominate ($L \leq 15\text{ B}$) and live duplication is low ($k < 1.5$), Conventional hash tables are optimal.
- 2) **Large-Scale Embedded Workloads:** When scale and identifier lengths are large ($L > 15\text{ B}$, $N > 200\text{K}$), adaptive front-coding (SymTabV3) achieves significant physical RAM savings (20.2% / 13.52 MB on ZEPHYR).

IX. LIMITATIONS

1) Front-coded block decompression introduces measured reconstruction/copy overhead during tail lookups; 2) Global pool strings persist for compilation unit lifecycles; 3) 64-bit hash fingerprints carry a theoretical collision probability of $\approx 4 \times 10^{-10}$ at 260,000 symbols.

X. CONCLUSION

On the large ZEPHYR workload, SymTabV3 reduced physically measured final heap by 20.2% (13.52 MB) relative to the embedded conventional baseline. This reduction was achieved with sub-microsecond median lookup latency, while the p_{95} latency increased from $0.125 \mu s$ to $0.228 \mu s$. Thus the architecture demonstrates a measurable RAM advantage in the large-scale embedded regime, but does not satisfy the $1.25 \times p_{95}$ latency constraint.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *IEEE/ACM CGO*, 2004, pp. 75–86.
- [3] S. Meyers, *Effective STL*. Addison-Wesley, 2001.
- [4] H. Sutter, *Exceptional C++ Style*. Addison-Wesley, 2004.
- [5] J. Gosling, B. Joy, G. Steele, G. Bracha, and A. Buckley, *The Java Language Specification (Java SE 8 Edition)*. Addison-Wesley, 2014.
- [6] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. CreateSpace, 2009.
- [7] I. H. Witten, A. Moffat, and T. C. Bell, *Managing Gigabytes: Compressing and Indexing Documents and Images*. Morgan Kaufmann, 1999.
- [8] N. R. Brisaboa, R. Cánovas, S. Ladra, and G. Navarro, “Compressed string dictionaries,” in *SPIRE*, 2011, pp. 136–148.
- [9] M. A. Martínez-Prieto, N. R. Brisaboa, R. Cánovas, A. Fariña, and G. Navarro, “Practical compressed string dictionaries,” *Information Systems*, vol. 56, pp. 73–108, 2016.
- [10] P. Ferragina and G. Manzini, “Opportunistic data structures with applications,” in *IEEE FOCS*, 2000, pp. 390–398.
- [11] R. Grossi and J. S. Vitter, “Compressed suffix arrays and suffix trees with applications to text indexing and string matching,” *SIAM J. Comput.*, vol. 35, no. 2, pp. 378–407, 2005.

- [12] F. C. Botelho, R. Pagh, and N. Ziviani, "Simple and space-efficient minimal perfect hash functions," *ACM TALG*, vol. 5, no. 2, pp. 1–24, 2009.
- [13] D. R. Hanson, "Fast allocation and deallocation of memory structures in C programs," *Software: Practice and Experience*, vol. 10, no. 4, pp. 307–313, 1980.
- [14] S. P. Reiss, "Practical lexical scoping and symbol tables," *CACM*, vol. 26, no. 6, pp. 429–435, 1983.
- [15] P. Celis, "Robin Hood Hashing," Ph.D. dissertation, University of Waterloo, 1986.
- [16] R. Pagh and F. F. Rodler, "Cuckoo hashing," in *ESA*, 2001, pp. 121–133.
- [17] N. Askitis and R. Sinha, "HAT-trie: A cache-conscious trie-based data structure for strings," in *ALENEX*, 2007, pp. 97–109.
- [18] S. Heinz, J. Zobel, and H. E. Williams, "Burst tries for fast string access," *ACM TOIS*, vol. 20, no. 2, pp. 192–223, 2002.